

What's new

Apply-Agent — v0.1.0 update

First public preview

This update adds a **Chrome extension** for applying to jobs by hand, and a way to get notified the moment a new version ships. Your original setup guide (the automatic apply bot) follows this page and works exactly as before.

New in this version

- **Chrome extension — apply by hand, fast.** Pick your resume/persona, and it autofills the application on a company's site (Greenhouse, Lever, Ashby, Workable, and more). You can reword any answer in plain English with AI, then submit it yourself. Full walkthrough in “**The Chrome extension**” at the end of this guide.
- **Everything in the original guide still works the same** — the auto-apply bot is unchanged.

Get every update automatically

Turn on release notifications once, and GitHub tells you each time a new version is out — no need to check back.

1. Watch the project for releases (recommended)

- Open github.com/fgbosibo17/Apply-Agent.
- Click the **Watch** button (top-right) → **Custom** → check **Releases** → **Apply**.
- GitHub now emails you every time a new version ships.

2. Or subscribe by RSS — add this feed to any reader:

```
https://github.com/fgbosibo17/Apply-Agent/releases.atom
```

See all changes any time on the live updates page:

- fgbosibo17.github.io/Apply-Agent

When you get an update, refresh your copy — open a terminal in the project folder and run:

```
cd Apply-Agent
git pull
```

If you use the Chrome extension, also open `chrome://extensions` and click **Reload** on the Job Apply Agent card so it picks up the new version.

Apply-Agent

Setup guide

This agent finds remote jobs and applies to them for you. You don't need to code. This guide walks through setup once, and how to run it after that.

Project: github.com/fgbosibo17/Apply-Agent

Questions? Email fope.gbosibo@gmail.com — see Contact at the end for more.

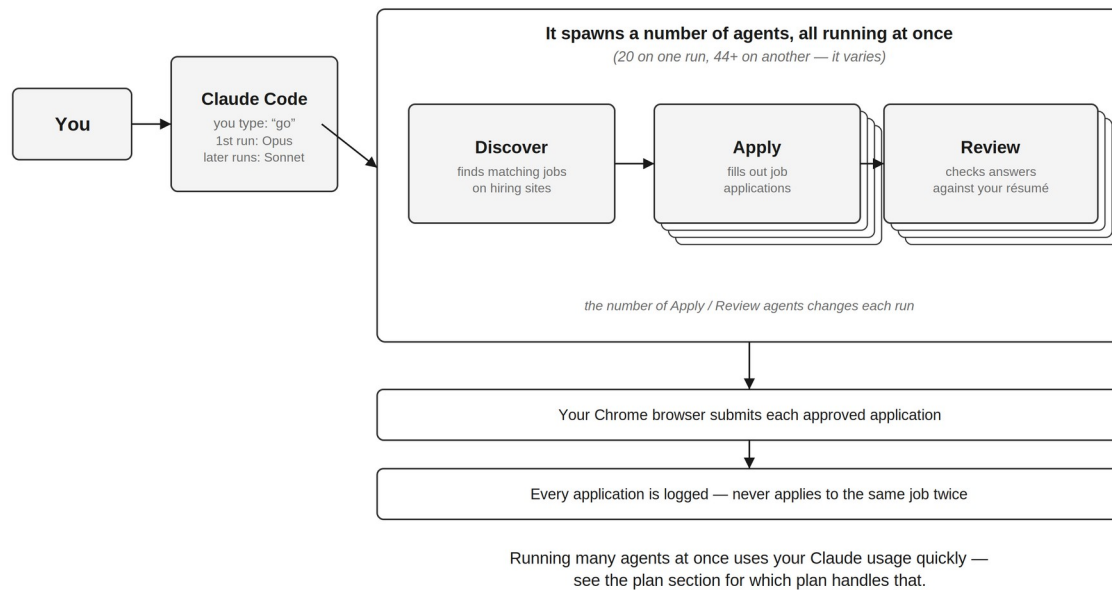
Contents

- How it works
- What you need
- 1. Get a mouse jigglers
- 2. Pick a Claude plan
- 3. Install Git, Node, and Claude Desktop
- 4. Download the project and add your resume
- 5. Open the project
- 6. Turn on Auto mode
- 7. First run (Opus)
- 8. Let it run
- 9. Later runs (Sonnet)
- 9a. Just apply - a numbers-game prompt
- Switching models
- 10. Leaving the house: Remote Control
- 11. Customizing it, and handling email login codes
- Notes on safety
- Can I use this for free, without a Claude plan?
- Cheat sheet
- Contact

How it works

You give it a goal in plain English. It reads your resume, searches real hiring sites, and spawns a number of agents to handle the work in parallel - how many depends on the run. One run might use 20, a busier one 44 or more. Three kinds of agents do the work: one finds jobs, others fill out applications, others check each answer against your resume before anything is submitted.

How Apply-Agent runs a session



1. **You give it a goal** - type "go," or paste a goal like "apply to 50 jobs."
2. **It spawns agents** - Claude Code runs a number of helpers at once. The count varies by run.
3. **They split the work** - Discover agents find jobs. Apply agents fill out applications. Review agents check the answers against your resume and correct or skip anything untruthful.
4. **It submits and logs** - approved applications go out through your own Chrome browser, on the company's real careers page. Every one is logged, so it never repeats a job.

Because it runs several agents at once, it uses your Claude usage quickly. That's the reason to pick a higher plan - see the next section.

What you need

- **A computer that can stay on** - Windows, Mac, or Linux, about 2 GB free, a steady internet connection.
- **Your resume** - PDF or Word. One is enough to start.

- **An email address and a LinkedIn account** - The email goes on your applications. LinkedIn is one of the sites it searches.
- **A Claude subscription** - Covered in step 2 - get the \$100 or \$200 plan.
- **A mouse jigglers** - Covered in step 1.

1. Get a mouse jigglers

The agent runs unattended for a long stretch - the first run alone can take around 30 minutes before it submits anything. If your computer sleeps or locks, it stops. A mouse jigglers keeps the cursor moving so that doesn't happen.

Get a plug-in, driver-free one - no software to install. [Example on Amazon](#)

Also worth doing: set your computer to not sleep while plugged in. Windows: Settings > System > Power. Mac: System Settings > Battery.

2. Pick a Claude plan

Running many agents at once uses your Claude usage fast. On the \$20 plan you'll likely hit the limit mid-run.

Plan	Price	Usage vs Pro	Notes
Pro	\$20/mo	1x	Will likely run out mid-session. Fine only for a small test.
Max 5x	\$100/mo	5x	Enough headroom for a normal run. Recommended starting point. Includes Auto mode (step 6).
Max 20x	\$200/mo	20x	Better if you run large batches often or in parallel. Also includes Auto mode.

The free plan won't work - it doesn't include Claude Code. Prices as of 2026; check claude.com/pricing for current numbers.

Recommendation: start with Max 5x (\$100/mo). Move to Max 20x if you're running large batches daily.

3. Install Git, Node, and Claude Desktop

One-time setup. Skip anything you already have.

- **Git** - downloads the project. git-scm.com/downloads
- **Node.js** - runs the project. Get the LTS version at nodejs.org
- **Claude Desktop** - the app you'll talk to. claude.ai/download, then sign in.

Install each with the default options. Then open a terminal - Windows: press the Windows key, type PowerShell, Enter. Mac: press Cmd+Space, type Terminal, Enter - and check each one:

```
git --version
node --version
npm --version
```

Each should print a version number. If one says “not recognized,” close the terminal and open a new one.

4. Download the project and add your resume

In the terminal:

```
git clone https://github.com/fgbosibo17/Apply-Agent.git
cd Apply-Agent
npm install
npx playwright install chrome
```

Prefer not to use the terminal? Open [the project page](#), click Code > Download ZIP, and unzip it. You'll still need to run the last two lines above inside that folder.

Drop your resume (PDF or Word) into the Resume folder inside the project. Any filename works.

5. Open the project

Open it in its own window. Two options, same result:

Option A: an IDE (e.g. VS Code)

1. Open the **Apply-Agent** folder in VS Code, or any code editor you already use.
2. Open its built-in terminal and type **claude** and press Enter. That starts Claude Code inside the editor.

Option B: Claude Desktop

1. Open the Claude Desktop app.
2. Go to the **Code** tab inside Claude Desktop — this is Claude Code running inside the app, no separate editor needed.
3. Open the **Apply-Agent** folder you downloaded.

Either way you end up talking to the same Claude Code — pick whichever you're more comfortable with.

6. Turn on Auto mode

By default Claude Code asks permission before each action. This agent takes hundreds of small steps, so switch to a mode that doesn't stop and ask every time.

Press **Shift + Tab** to cycle modes. The current mode shows at the bottom of the window. Keep pressing until it says:

```
Auto
```

The cycle order is: normal → accept edits on → plan mode → Auto → back to normal.

Don't stop at “**accept edits on**” — that mode still pauses to ask before running commands (like opening a browser or applying to a job). **Auto** mode is the one that runs the whole session without stopping to ask, with a background safety check. It's available on both Max 5x and Max 20x (not on Pro).

7. First run (Opus)

Use **Opus** for the first run - it does the heavier work of reading your resume and building your profile.

```
/model opus
```

Fill in the blanks below and delete lines that don't apply, then paste it in.

After pasting, click at the start of `/goal` and re-type it so it turns blue - that's how Claude Code recognizes it as a command.

```
/goal I added my resume(s) to the Resume/ folder. Read them, extract ALL my information, and build out my profile (fill src/personas.js and the CLAUDE.md APPLICATION ANSWERS from my resume). Then find REMOTE-ONLY jobs that match and apply to AT LEAST 50 jobs per resume.
```

```
MY INFO (use this; read the rest from my resume):
```

- Work authorization: << e.g. US citizen; do NOT need current or future sponsorship >>
- Gender: << e.g. Female >>
- Race/ethnicity: << e.g. Black or African American >> | Hispanic/Latino: << No >>
- Veteran status: << e.g. Not a veteran >> | Disability: << e.g. No >>
- LinkedIn: << your LinkedIn URL, or write "none" >>
- Location: << City, ST >> (remote only)
- Salary expectation: << e.g. Open/negotiable, OR a range like \$45k-\$60k >>
- Highest education: << e.g. High School Diploma / Bachelor's in X / none >>

- Certifications/licenses: << list them, or write "none" >>
- Job roles to apply for: << e.g. Medical Claims Specialist, Data Entry Clerk >>

HARD RULES - do not break these:

1. ASK ME before you start if ANYTHING is unclear or missing. On this FIRST run, DO NOT ASSUME - ask me instead. Only start once you have what you need.
2. REVIEW EVERY APPLICATION BEFORE YOU SUBMIT. Use the two-agent fill->review pipeline (workflows/fill-review-applications.js): one agent fills from my real questions, a second independent agent audits every answer against my resume and corrects/rejects before anything is submitted.
3. DO NOT FABRICATE. Genuinely think before answering. Never claim a degree, certification, license, skill, language, or years of experience I do not have. If I don't have it, the honest answer is "No" / "None" / "0".
4. Use resume blurbs GROUNDED IN MY RESUME for open-ended questions. No invented stories, metrics, or achievements that aren't on my resume.
5. DO NOT SUBMIT COVER LETTERS unless a form strictly REQUIRES one to submit.
6. REMOTE ONLY. Skip onsite/hybrid roles and roles that require a credential, license, or language I don't have.
7. Log everything and never apply to the same job twice.

When my profile is built and you've confirmed my answers with me, start applying.

- It reads your resume and asks a short list of questions - salary, work authorization, EEO answers. Answer honestly.
- Once your profile is set, it starts finding and applying to jobs, checking every answer first.
- If it hits a login page, you sign in yourself - it never types your password. The login is saved for next time.

8. Let it run

Plug in the mouse jiggler and keep the window open. The first run can take around 30 minutes before the first application goes out. You can step away.

To stop it, tell it to stop, or close the window.

9. Later runs (Sonnet)

After the first run, your profile is already built. Switch to **Sonnet** - faster and cheaper - for every run after that.

```
/model sonnet
```

Change << N >> to how many more jobs to apply to, then paste it in (re-type /goal so it turns blue).

```
/goal My profile is already set up. Discover MORE remote-only jobs that match my target roles and apply to at least << N >> more per resume, continuing from where I
```

left off (dedupe against seen-jobs.csv - never re-apply to a job I've already seen).

HARD RULES - do not break these:

1. REVIEW EVERY APPLICATION BEFORE YOU SUBMIT. Use the two-agent fill->review pipeline (workflows/fill-review-applications.js): one agent fills from the real questions, a second independent agent audits every answer against my resume and corrects/rejects before anything is submitted.
2. DO NOT FABRICATE. Genuinely think before answering. Never claim a degree, certification, license, skill, language, or years of experience I do not have. If I don't have it, the honest answer is "No" / "None" / "0".
3. Use resume blurbs GROUNDED IN MY RESUME for open-ended questions. No invented stories, metrics, or achievements.
4. DO NOT SUBMIT COVER LETTERS unless a form strictly REQUIRES one to submit.
5. REMOTE ONLY. Skip onsite/hybrid roles and roles that require a credential, license, or language I don't have - the review agent should reject those.
6. Don't re-ask my setup info - it's already in src/personas.js and CLAUDE.md. Only ask me if something genuinely new comes up.

Report how many you submitted per resume when you're done.

It picks up where it left off and skips anything it's already seen.

This is the full, careful mode - it fills and then independently reviews every answer before submitting. That's quality over quantity, and it costs more tokens per application because two agents look at every job. If you're trying to get a high volume of applications out and want to watch your usage, use 9a below instead, or mix both: run 9 for roles you care most about, 9a for the rest.

9a. Just apply - a numbers-game prompt

A lighter version: one agent applies without the separate review pass. It still uses your real resume facts and won't invent qualifications - it just skips the second independent check, which is the part that costs the most tokens. Use this when you want more applications out for the same usage.

/goal My profile is already set up. Discover MORE remote-only jobs that match my target roles and apply to at least << N >> more per resume, continuing from where I left off (dedupe against seen-jobs.csv - never re-apply to a job I've already seen).

MODE: numbers game - apply directly, skip the separate review-agent pass.

HARD RULES - do not break these:

1. USE ONE AGENT for this run. Do not spawn multiple parallel agents - I'm being mindful of token usage this time, so work through jobs one at a time yourself instead of fanning out.
2. APPLY, DON'T DOUBLE-REVIEW. Fill and submit directly - skip the separate independent review-agent step this time to save tokens. You still have to genuinely think about each answer before submitting; this only removes the second agent's re-check, not the thinking.
3. STAY GROUNDED IN MY RESUME. Never claim a degree, certification, license, skill, language, or years of experience I do not have. If I don't have it, the honest answer is "No" / "None" / "0". This rule does not relax in this mode.

4. Use resume blurbs grounded in my resume for open-ended questions. No invented stories, metrics, or achievements.
5. DO NOT SUBMIT COVER LETTERS unless a form strictly requires one.
6. REMOTE ONLY. Skip onsite/hybrid roles and roles that require a credential, license, or language I don't have.
7. Log everything, same as always.

Report how many you submitted per resume when you're done.

Because there's no review agent double-checking, spot-check applications-log.csv a bit more often when you use this mode.

Switching models

Type `/model` any time to change which model runs the agent.

Command	What it is	Use for
<code>/model opus</code>	Most capable	First-time setup, unusual application forms.
<code>/model sonnet</code>	Fast, balanced	Everyday runs after setup.
<code>/model haiku</code>	Fastest, cheapest	High-volume runs where cost matters most.
<code>/model opusplan</code>	Hybrid	Opus plans, Sonnet executes.

10. Leaving the house: Remote Control

This deserves its own explanation, because it's the difference between being stuck at your desk for hours and actually walking away while the agent keeps working.

What it is

Remote Control is a feature built into Claude Code that connects your phone (or any browser) to a session that's running on your computer. It's not a separate mobile version of the agent - it's a live window into the exact same session, so anything happening in the terminal shows up on your phone in real time, and anything you type on your phone shows up in the terminal.

How it works

The agent itself never leaves your computer. Your files, your browser, your logins, everything the agent is doing to apply for jobs - all of that keeps running locally on your machine, exactly as it would if you were sitting in front of it. Your phone doesn't do any of the work; it just opens a secure connection to Anthropic's servers, which relay messages back and forth between your phone and your computer. That's why it needs your computer to stay on: turn the computer off, and there's nothing on the other end for your phone to connect to.

How to turn it on

Inside the session, type `/remote-control` (or the shorthand `/rc`). A QR code appears in the terminal - scan it with your phone's camera, and it opens directly into the same session. You can also open the **Code** tab in the Claude mobile app, where any Remote Control sessions you've started will be listed.

What you can do once it's connected

- Watch progress live - see applications going out as they happen, the same output you'd see in the terminal.
- Approve or answer anything it asks - if it needs a decision from you, you can respond from your phone instead of walking back to your desk.
- Send it new instructions mid-run - tell it to prioritize a certain company, skip a role, or wrap up for the day.

Important: Remote Control does not let the work continue if your computer actually falls asleep. If the laptop sleeps, everything pauses until it wakes back up, and it will not reconnect on its own. **This is exactly why the mouse jigglers from step 1 matters:** keep it plugged in and the computer will never sleep, so you can genuinely leave the house, check progress on your phone, and come back to a session that's kept applying the whole time.

11. Customizing it, and handling email login codes

This is a project you own, not a finished product you're renting. That's the real difference between this and a paid job-application service: those tools ship one fixed workflow and you adapt to it. Here, the

prompts, the persona file, and the answer bank are plain text you can open and edit, so the agent adapts to you instead - a different tone in your cover letters, a stricter salary floor, a new ATS it doesn't handle yet. If something about your job search is unusual, you change the project to fit it rather than working around a locked-down tool.

Handling sites that email you a login code

Some job sites and ATS platforms verify you by emailing a one-time code instead of a normal password. Left alone, the agent will stop and ask you for that code every time. You can instead let it check your email itself, the first time you set it up, using a prompt like this:

```
This site is asking for an email verification code to log in.  
My email is << your email >> and I want you to handle this from  
now on: open a browser tab, let me log into Gmail myself (I'll  
type the password - don't ask me for it or type it yourself),  
and once I'm logged in, save this browser session the same way  
you save my other logins. From now on, when a site emails me a  
code, check that Gmail tab for the latest message, read the  
code, and enter it - don't stop and ask me for it unless you  
can't find it.
```

You still log into Gmail yourself in the browser window - the agent never sees or types your password. Once that session is saved, it can read the code out of an email the same way it reads anything else on a page you've given it access to. If it ever can't find the code, it should still fall back to asking you.

Notes on safety

- These are real applications to real employers. Only apply to jobs you'd take, and keep answers truthful - the review step helps, but you're responsible for what goes out.
- It never types passwords. When a login is needed, you enter it yourself in the browser.
- It won't apply to the same job twice. Every application is logged in applications-log.csv inside the project folder.
- It only submits on a company's real careers page, never through a job board's "Easy Apply."
- If it hits something it can't handle safely - a CAPTCHA, a password, anything odd - it stops and asks you.

Can I use this for free, without a Claude plan?

Short answer: no, not with Claude itself. If you're technical and want to swap in a different model that you can run for free, it's possible - see below.

Why Claude isn't a free option

Claude's models (Sonnet, Opus, Haiku, Fable) are closed-weight and proprietary. Anthropic doesn't release them for local or self-hosted use, free or paid. The only ways to run Claude at all are Anthropic's own API, or paid cloud access through AWS Bedrock, Google Vertex AI, or Microsoft Foundry - every one of those is metered, and none of them are free.

The free alternative: a different, open-weight model

If you want \$0, you're not running Claude - you're running a different model entirely, on your own hardware, through a tool like Ollama or LM Studio. A few solid free options as of 2026:

- **Qwen2.5 / Qwen3 (7B-32B)** - currently the strongest open-weight models for coding and reasoning at sizes a regular computer can run.
- **DeepSeek-Coder-V2 / DeepSeek-R1, distilled (7B-32B)** - good reasoning, free, runs locally.
- **Llama 3.1 / 3.3 (8B-70B)** - Meta's model, solid general purpose, the biggest community around it.
- **Mistral / Mixtral** - fast, decent quality, lighter on RAM.

Realistic hardware: a 7B-14B model runs on most modern laptops (16GB+ RAM, ideally a GPU with 8GB+ VRAM). A 32B+ model needs a real GPU (24GB+ VRAM) or it'll crawl on CPU.

Local vs. renting a GPU on AWS

- **Local (actually free)** - Ollama on your own machine costs nothing beyond electricity. This is the only genuinely free path.
- **Self-hosting on AWS (not free)** - you'd rent a GPU instance (e.g. EC2 g5/g6) to run the same open-weight models. You pay by the hour for the GPU, often more than a hosted API would cost, plus you take on the setup, uptime, and security yourself.

If you want the tradeoffs explained well, watch [this video](#) on open-source vs. paid models - it covers why plenty of people still choose to pay for Claude even with free options on the table.

Cheat sheet

When	Do this
Before any run	Plug in the mouse jiggler. Make sure the computer won't sleep.
Open it	Claude Desktop > Code > open the Apply-Agent folder (or open the folder in an IDE and type "claude" in its terminal).
Stop the prompts	Shift+Tab until you see "Auto."
First run only	/model opus > paste the first-run prompt > re-type /goal so it's blue.
Every run after (careful)	/model sonnet > paste the 9. prompt with N set > re-type /goal.
Every run after (volume)	/model sonnet > paste the 9a. prompt with N set > re-type /goal.
Leave the house	/remote-control (or /rc), scan the QR code with your phone. Mouse jiggler still has to stay in - the computer just can't sleep.
Check your applications	Open applications-log.csv in the project folder.

Contact

Questions, stuck on a step, or found something in this guide that's wrong? Email me at fope.gbosibo@gmail.com and I'll help where I can.

If this guide saved you time and you'd like to support the work, an optional donation is always appreciated, never expected: ko-fi.com/fopeg

New section

The Chrome extension

Apply to a job by hand, with autofill + AI answer help

This is a browser extension for when **you** apply to a job yourself, in your own Chrome. It fills the boring fields so you're not retyping everything, lets you fix any answer with AI, and then you click submit. It's separate from the automatic bot in the rest of this guide.

Which one should I use?

- **The auto bot** (rest of this guide): hands-off. It finds jobs and applies in bulk on its own.
- **The extension** (this section): you found a job yourself and want help filling it out fast — including on sites the bot skips (logins, CAPTCHAs), because you're right there to handle them.

One-time setup

1. Load the extension into Chrome

- Open `chrome://extensions`.
- Turn on **Developer mode** (top-right).
- Click **Load unpacked** and select the extension folder inside the project.

2. Add your details — in the extension folder, copy `personas.example.js` to `personas.js` and fill in your name, email, phone, LinkedIn, and resume file paths. This file stays on your computer only.

3. Start the AI helper (only needed for the “refine” feature). In a terminal in the project folder, run this and leave it open:

```
npm run refine-helper
```

It uses your existing Claude login — **no API key**. If it says login is needed, run `claude` once in a terminal and sign in, then start it again.

Using it on a job

- 1 Open a job's **application page** (Greenhouse, Lever, Ashby, Workable, etc.).
- 2 Click the **Job Apply Agent** icon → pick the **persona** (resume) for that job.
- 3 Click **Fill only**. It fills the form. A resume-upload prompt (“started debugging this browser”) is normal — leave it.
- 4 A **Refine answers** panel appears. To reword an answer, type an instruction (“make this shorter”) and click  — Claude rewrites just that answer. You can also edit any field directly.
- 5 **Review, then click the site's Submit yourself.**
- 6 Click **Mark applied & log** (in the panel, or from the extension popup after you submit) so it's recorded and never repeated.

Good to know

- It **never types passwords** — you sign in yourself in the browser.
- It shares the same application log as the bot, so **nothing gets applied to twice**.
- Not on the Chrome Web Store yet — that's why it's "load unpacked." Once it is, updates become automatic.